**“Año de la Esperanza y el Fortalecimiento de la Democracia”**

**UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ**
FACULTAD DE INGENIERÍA DE SISTEMAS

# PRÁCTICA SEMANA 11

## Introducción a Django: Arquitectura MTV, URLs, Vistas (FBV/CBV), Plantillas (Herencia, Bloques, Tags, Filtros) y Modelos (ORM, Consultas)

**ESTUDIANTE**

Gago Uribe Edgar Robert

**DOCENTE**

Mg. Miguel Aguilar Coronación

**ASIGNATURA**

Desarrollo de Aplicaciones Web

**HUANCAYO - PERÚ**

**2026-II**

UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS

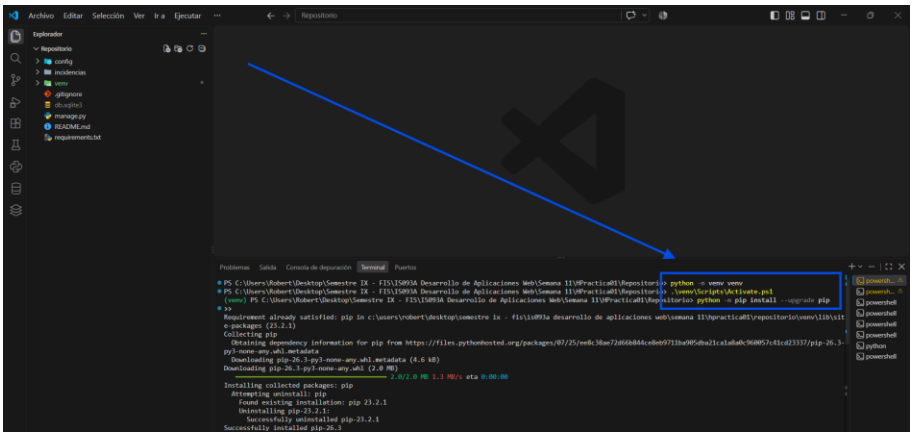
## OBJETIVO DE LA PRÁCTICA:

1. Construir una aplicación web funcional con Django 5.x, aplicando el patrón MTV (Model-Template-View), configurando enrutamiento con urls.py, desarrollando vistas basadas en funciones (FBV) y clases (CBV), implementando plantillas con herencia, bloques, etiquetas y filtros, y definiendo modelos con el ORM de Django para persistencia en SQLite. Validar el ciclo solicitud-respuesta, la migración de base de datos y preparar la estructura para formularios, admin y sesiones (Semana 12).

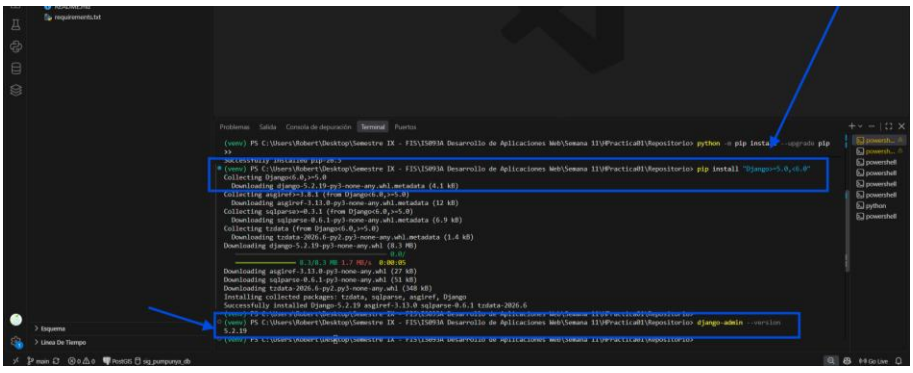
## EJERCICIO PASO A PASO

### 1. Crear entorno virtual, instalar Django, iniciar proyecto (startproject) y app (startapp). Verificar runserver y estructura de carpetas

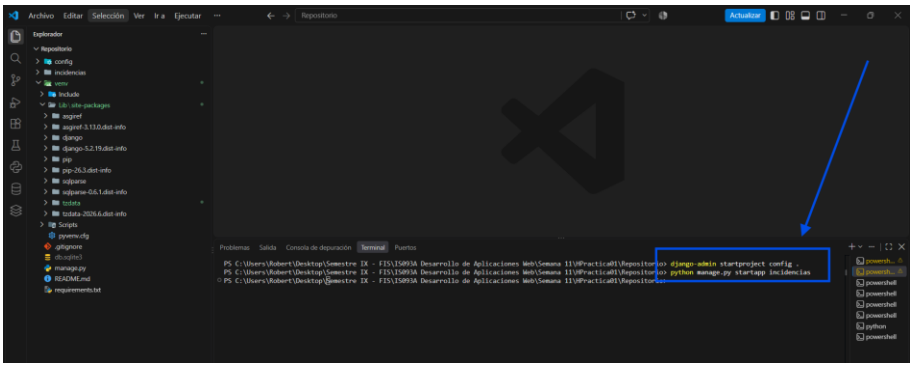
Creación de entorno virtual con python -m venv venv



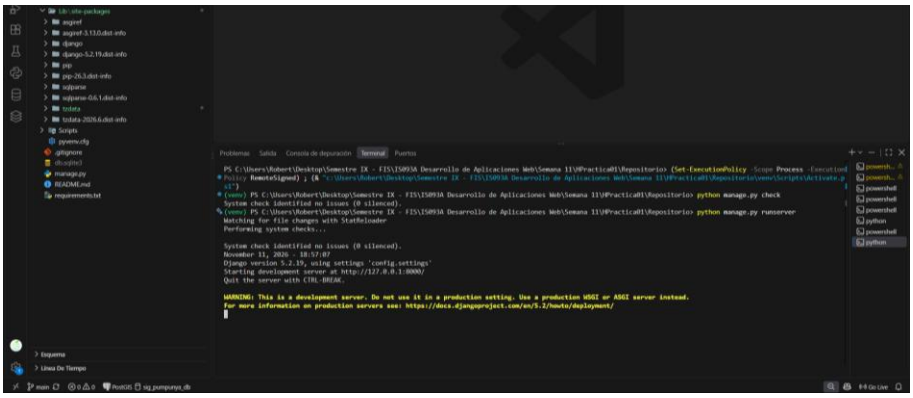
Instalación de Django con pip install



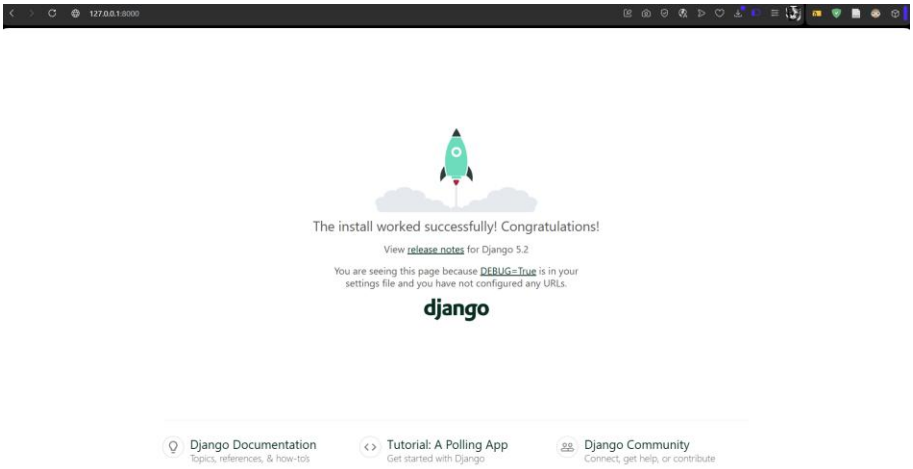
Iniciando proyecto y app



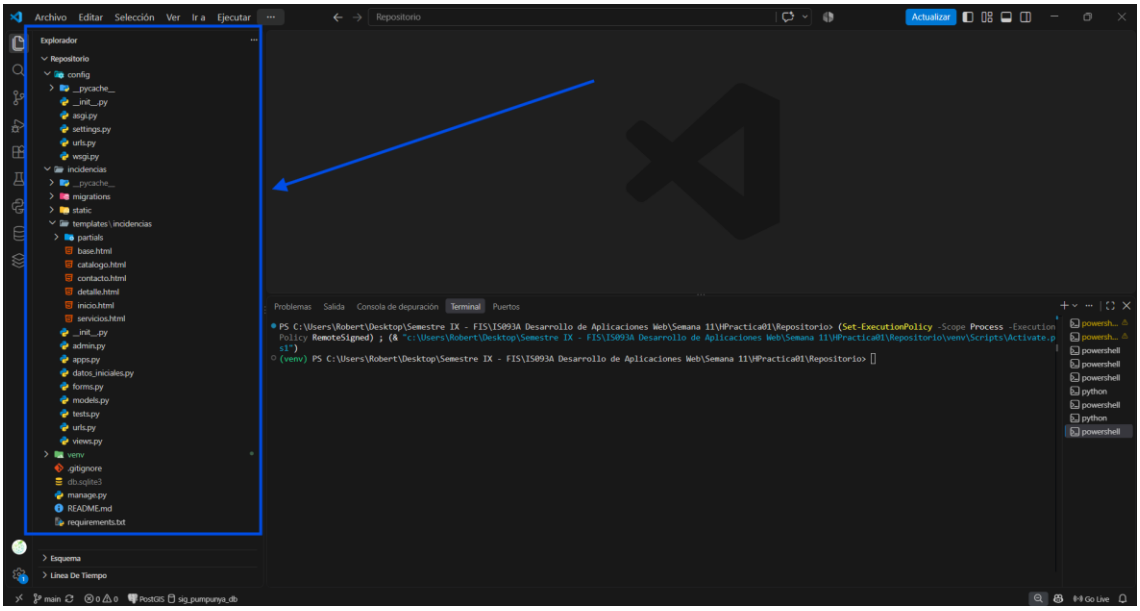
Verificando runserver con python manage.py check y python manage.py runserver



Página por defecto de Django en http://127.0.0.1:8000

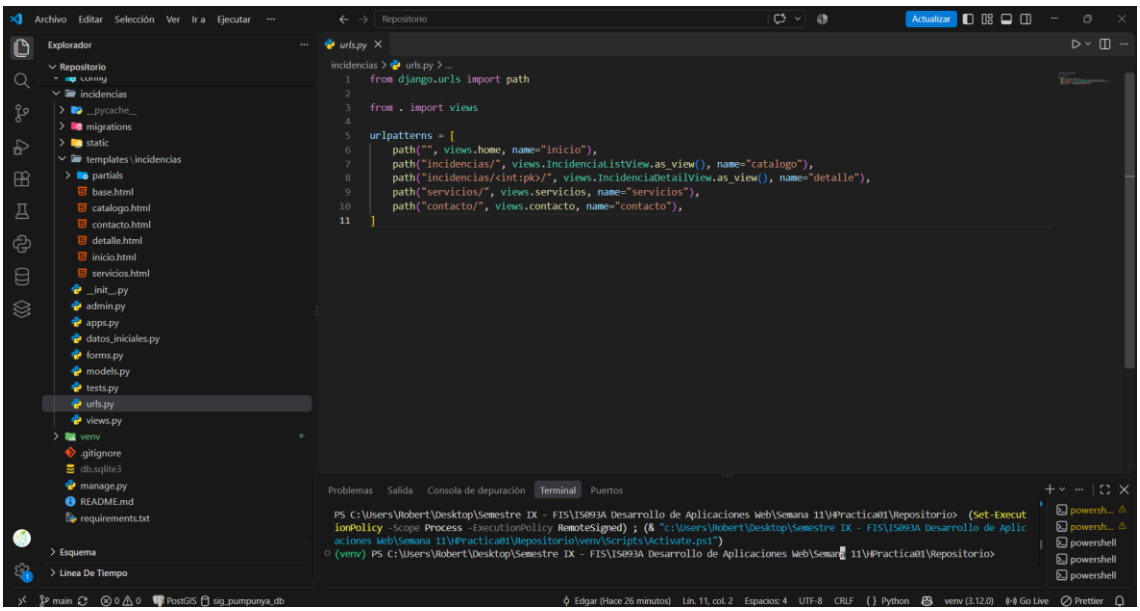


Estructura de carpetas

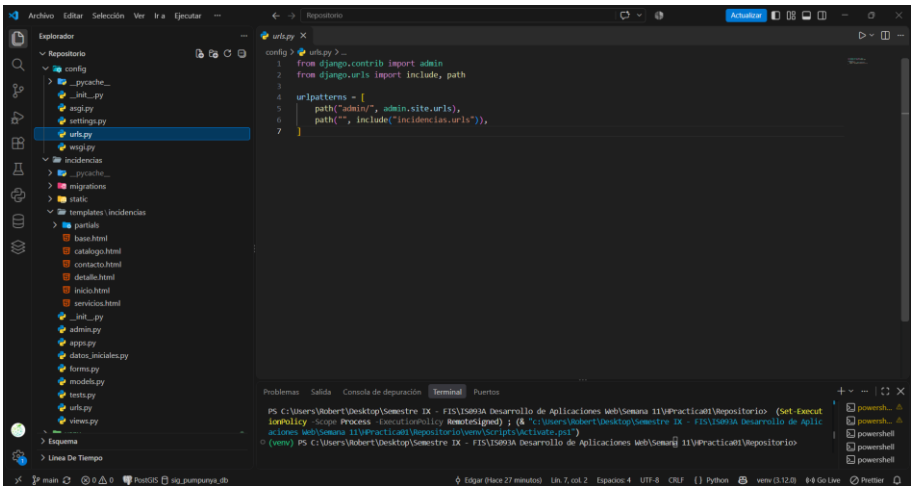


**2. Configurar rutas: app/urls.py, incluirla en project/urls.py. Mapear path() a vistas. Usar name para reversibilidad**

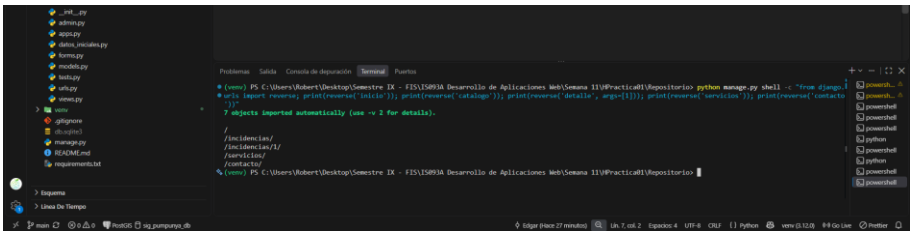
Archivo incidencias/urls.py que define las rutas de la app con path(), cada una asociada a una vista y con un name para poder referenciarla.



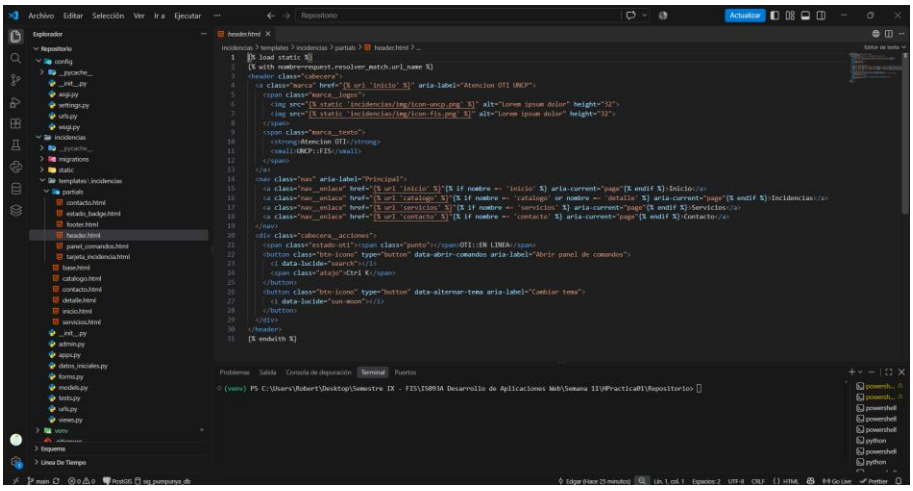
Archivo config/urls.py que incluye el enrutador de la app con include("incidencias.urls"), desacoplando las rutas de la app de las del proyecto.



Uso de reverse() con los nombres de las rutas. Se comprueba la reversibilidad de que cada name devuelve su URL correspondiente, incluida la ruta con parámetro <int:pk>

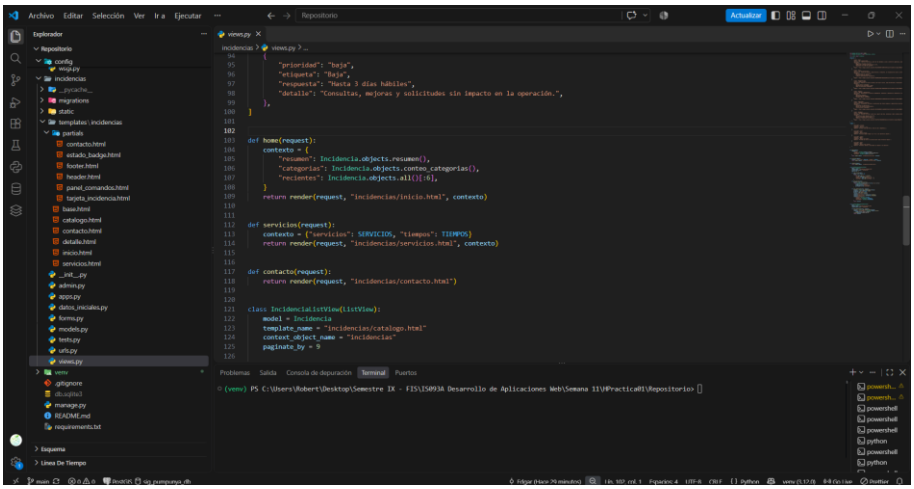


Las plantillas usan {% url 'nombre' %} en lugar de URLs escritas manualmente.

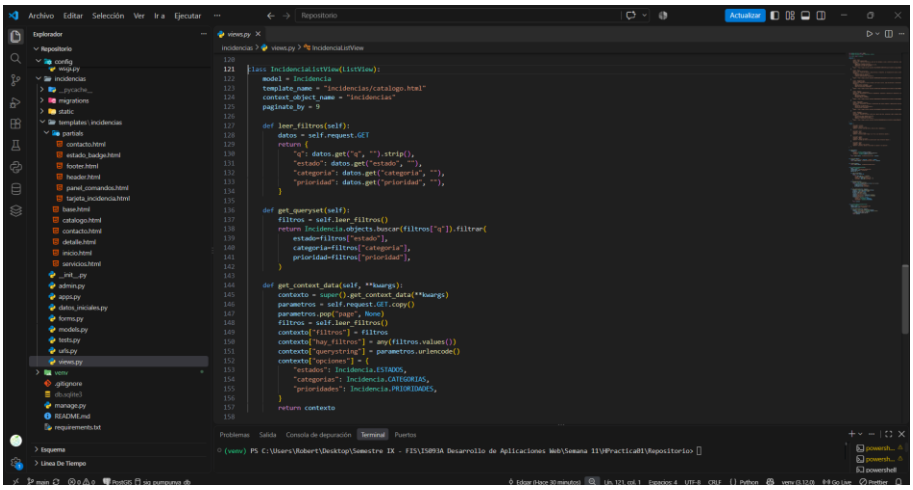


**3. Implementar vistas: 1 FBV (def home) y 1 CBV (ListView). Retornar HttpResponse o render(). Pasar contexto**

Aista basada en función (FBV) home que consulta el modelo mediante el ORM, arma el contexto de resumen, categorias, recientes y lo entrega a la plantilla con render()



Vista basada en clase (CBV) IncidenciaListView, que hereda de ListView. Con get_queryset() filtra según los parámetros de la URL y con get_context_data() agrega al contexto los filtros y las opciones.



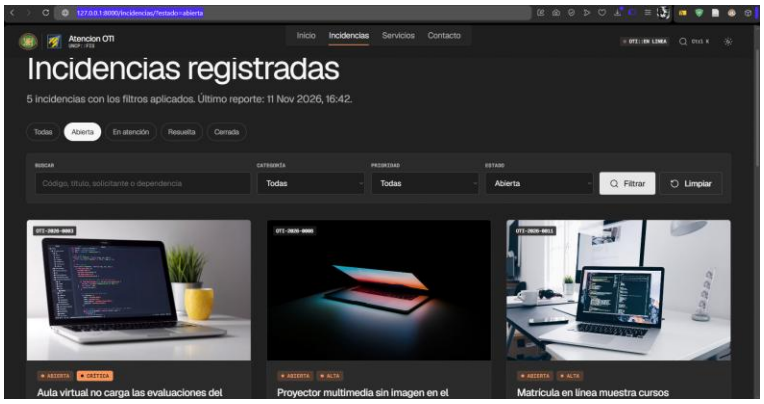
Resultado de la vista home en el navegador. Los datos mostrados de incidencias recientes y cifras de resumen provienen del contexto enviado por la vista.




Resultado de la vista IncidenciaListView con listado de incidencias usando paginación.

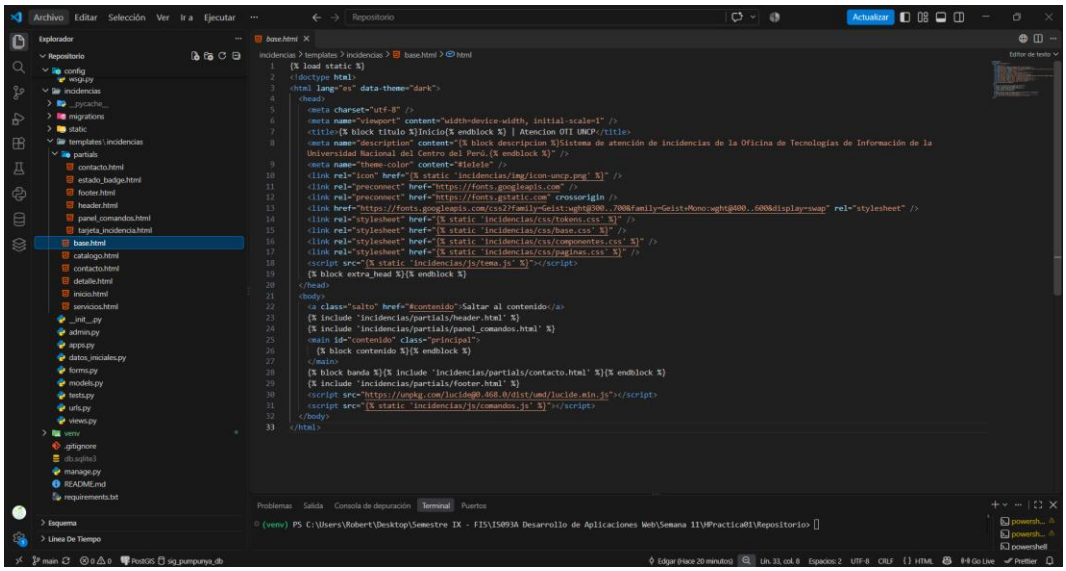



La misma vista con un parámetro en la URL get_queryset() devuelve solo las incidencias abiertas.

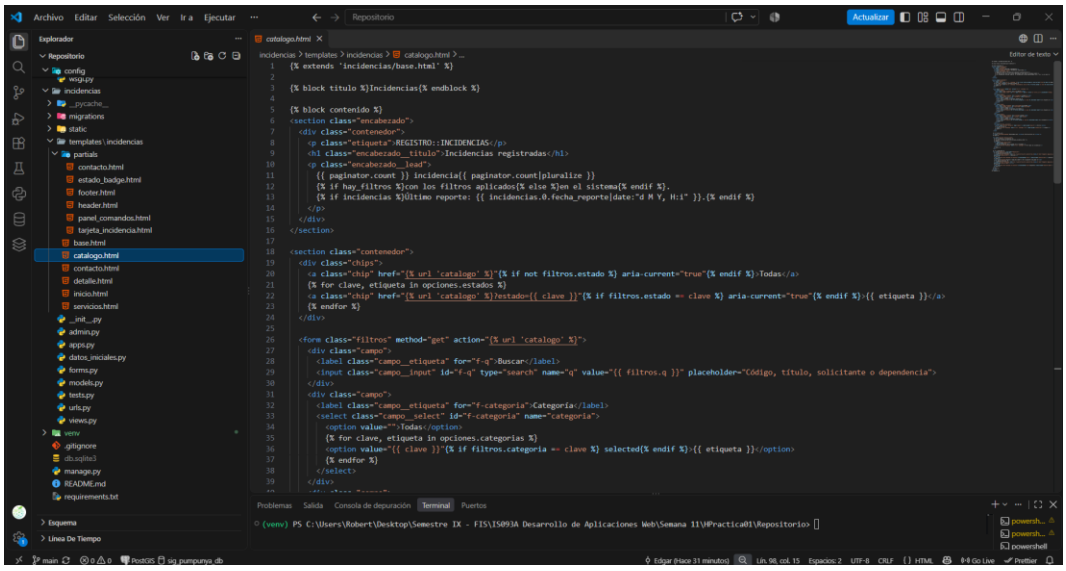


**4. Crear plantillas: base.html con bloques, catalogo.html con {% extends %}, {% for %}, {% if %}, filtros (`title, date`)**

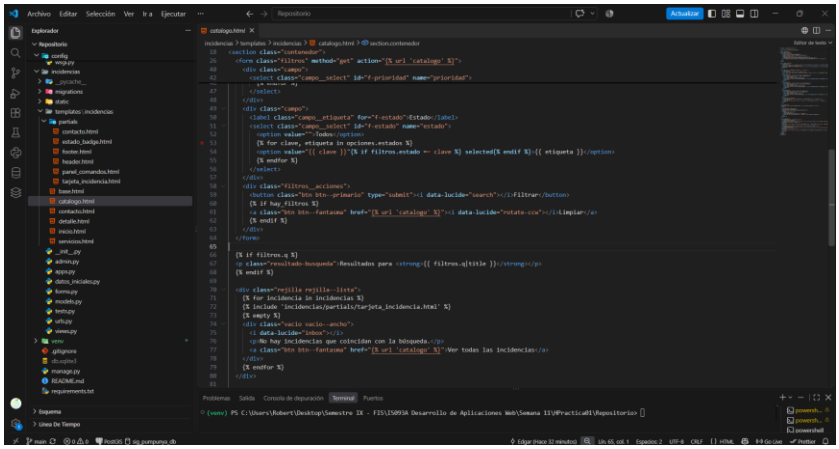
Plantilla base con los bloques titulo, contenido y banda. Incluye mediante {% include %} los fragmentos reutilizables de header, panel de comandos, contacto y footer.



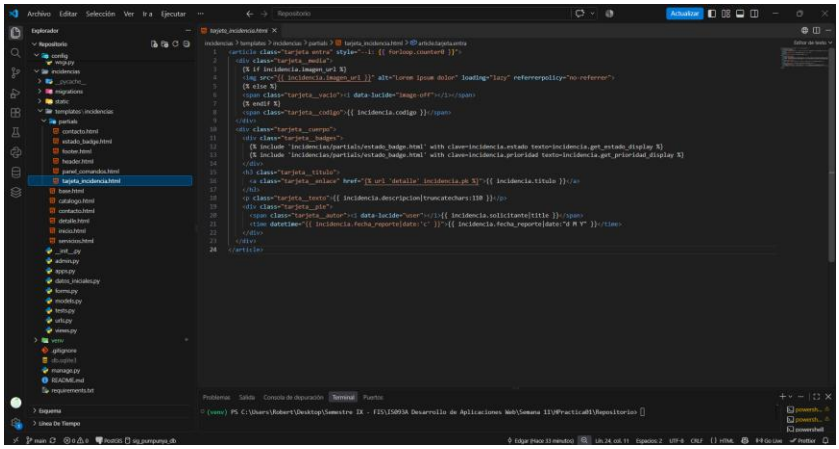
catalogo.html hereda de la plantilla base con {% extends %} y define su contenido dentro del bloque contenido.



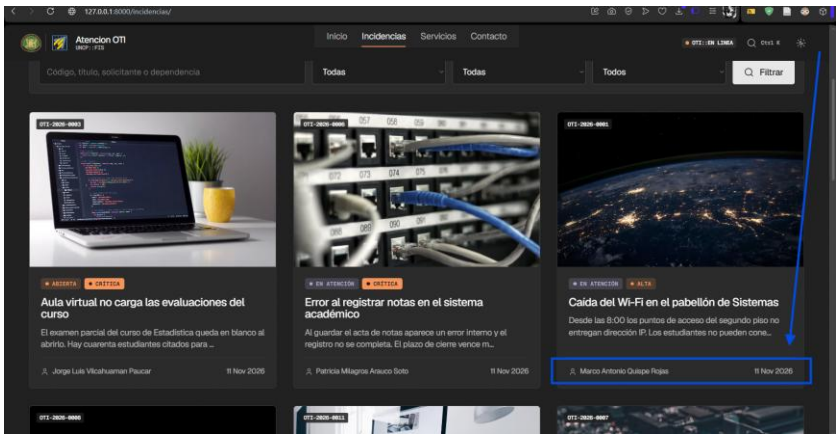
Uso de etiquetas de plantilla{% for %} que recorre las incidencias, {% empty %} cubre el caso sin resultados e {% if %} controla elementos condicionales



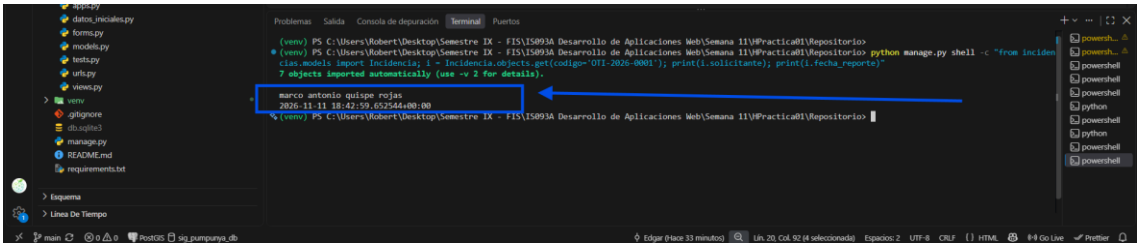
Uso de filtros de title que convierte el nombre del solicitante a mayúsculas iniciales y date da formato a la fecha de reporte.



Resultado de los filtros en el navegador que el solicitante aparece como “Marco Antonio Quispe Rojas” y la fecha como “11 Nov 2026”.



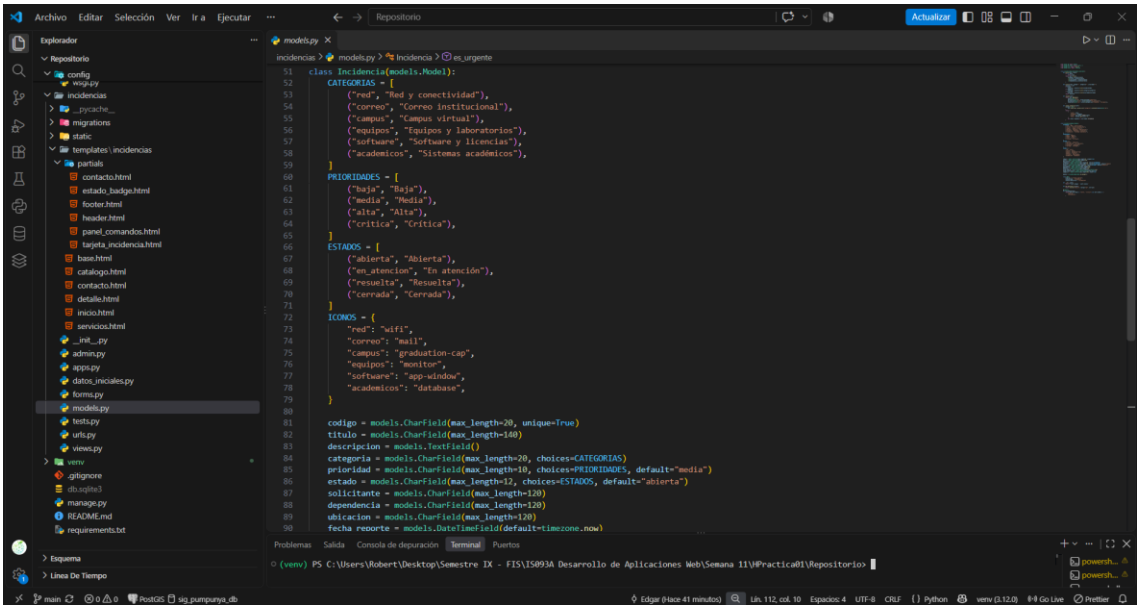
Mismo registro de incidencia pero presentado desde la base de datos, como se observa está con minúsculas y fecha completa, para contrastar con lo que muestran los filtros.



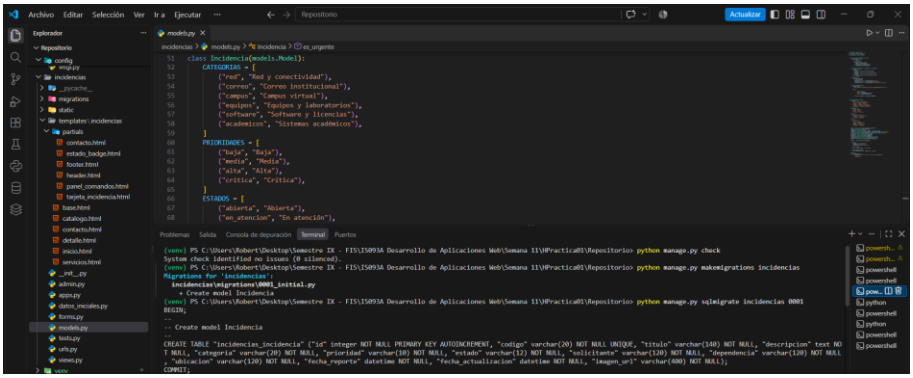

## 5. Definir modelo Producto, ejecutar makemigrations y migrate. Poblar datos, consultar con objects.all() y filter()

Para este caso, dado que la temática es un Módulo de Atención de Incidencias para la UNCP, el modelo Producto se implementó con nombre de Incidencia.

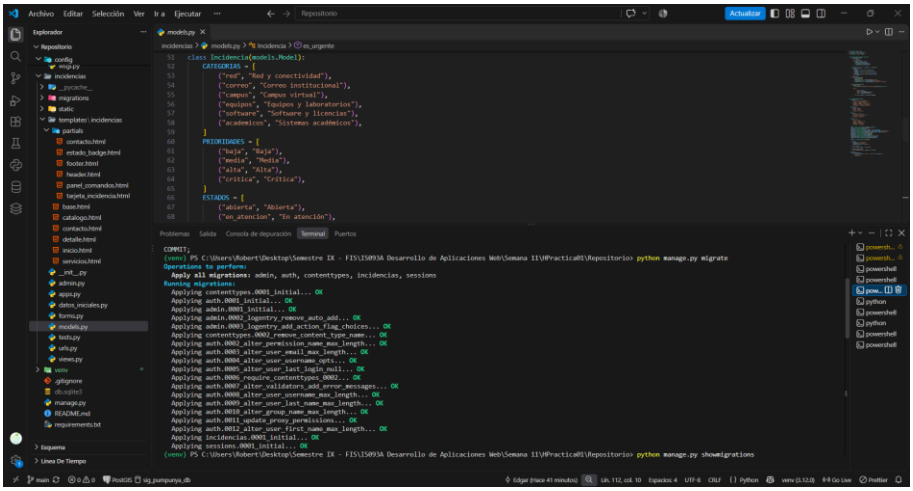
Modelo Incidencia con campos código, título, descripción, categoría, prioridad, estado, solicitante, dependencia, ubicación, fechas e imagen, Meta.ordering y el método __str__



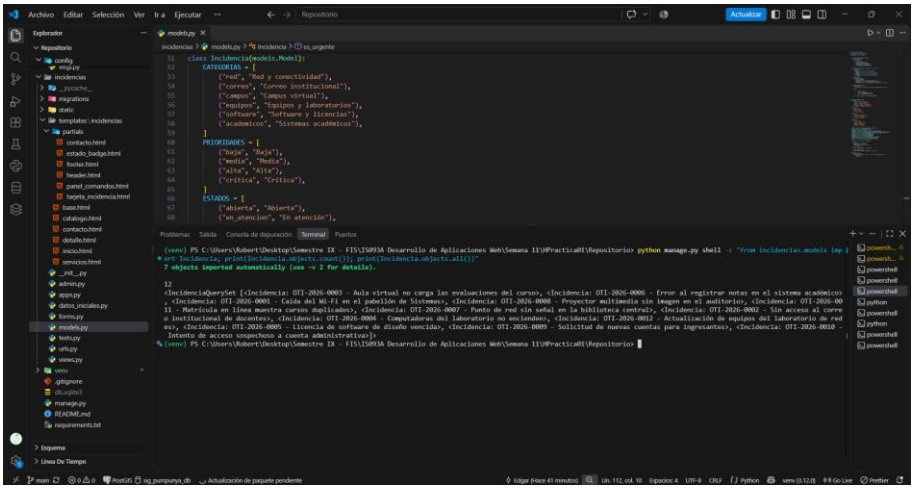

Ejecución de makemigrations incidencias y sqlmigrate que presenta el CREATE TABLE que el ORM genera a partir del modelo



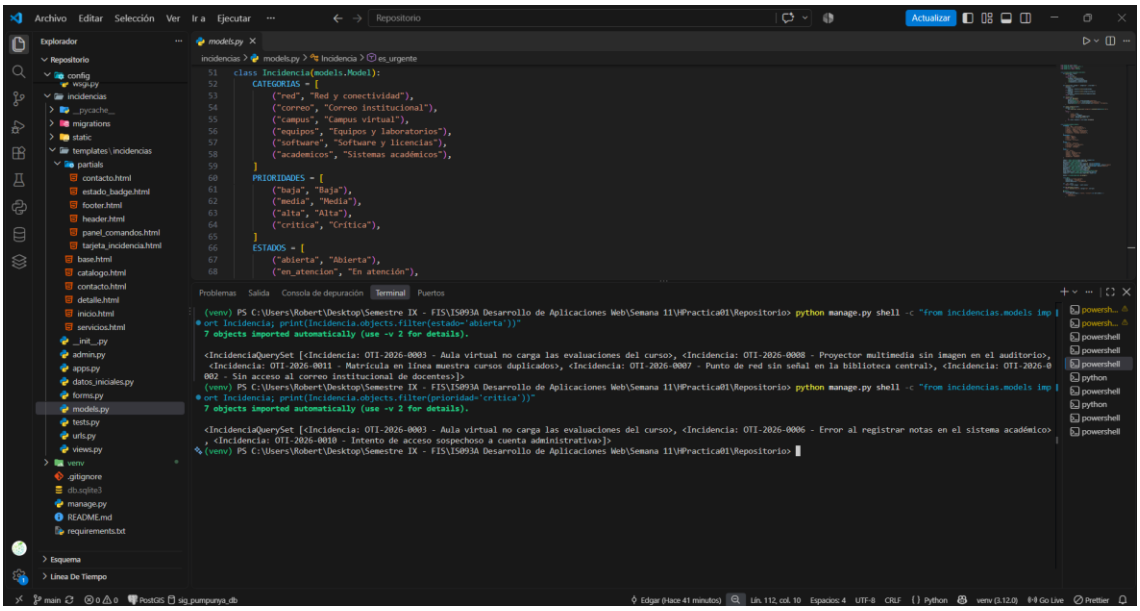
Poblado de datos mediante python manage.py migrate



Consulta con objects.all() devuelve las 12 incidencias, representadas por su __str__ con código y título.



Consultas con filter() para obtener las incidencias abiertas y las de prioridad crítica.

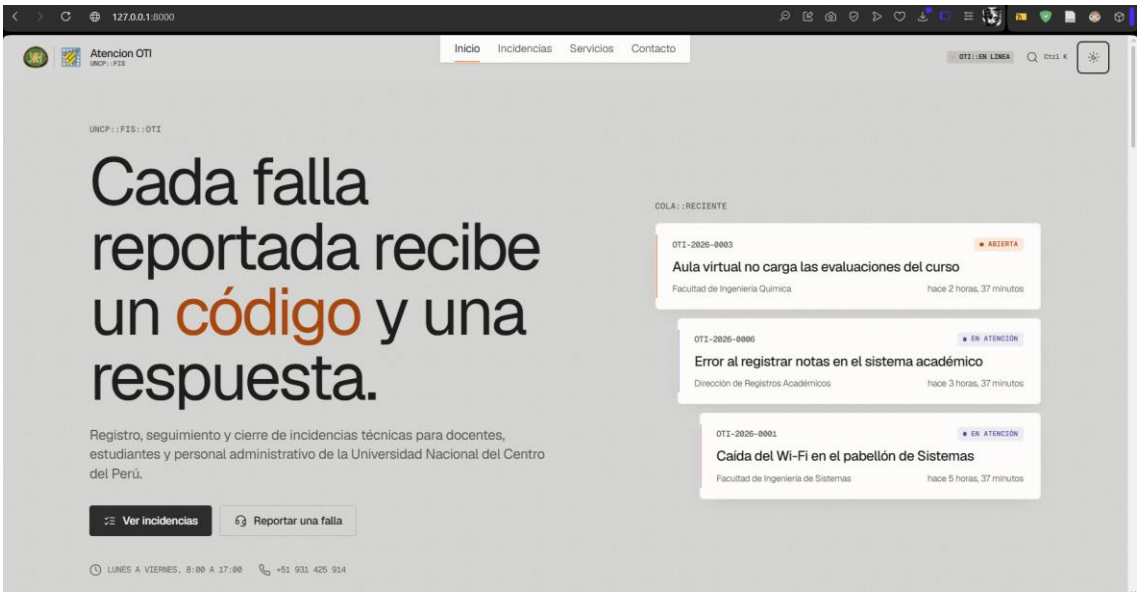


## CAPTURAS DE PANTALLA DE LA APLICACIÓN WEB

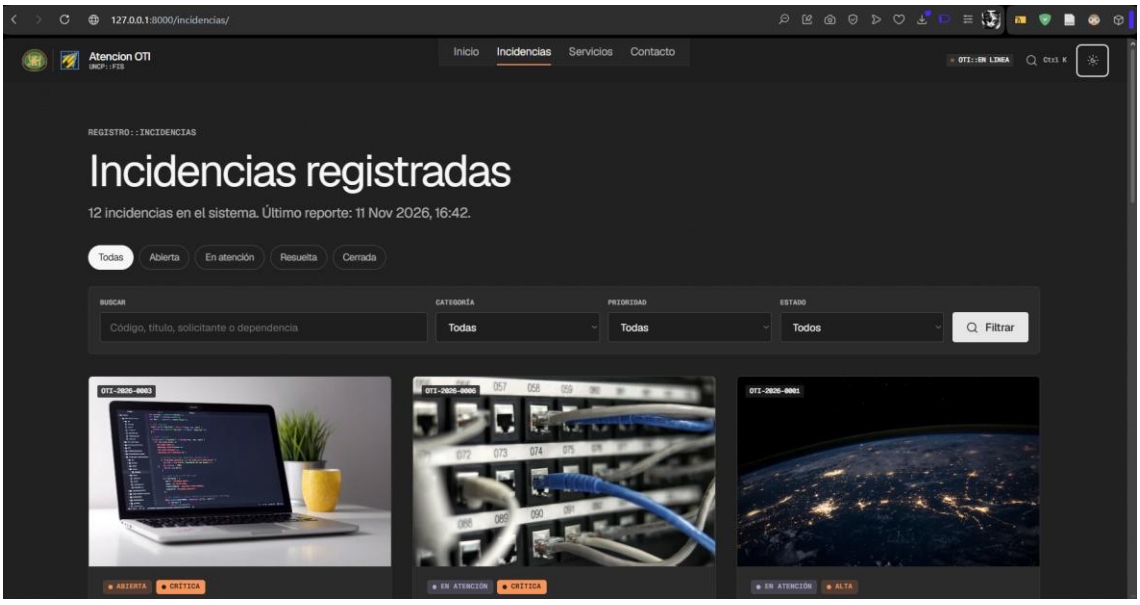
Inicio en Tema Oscuro



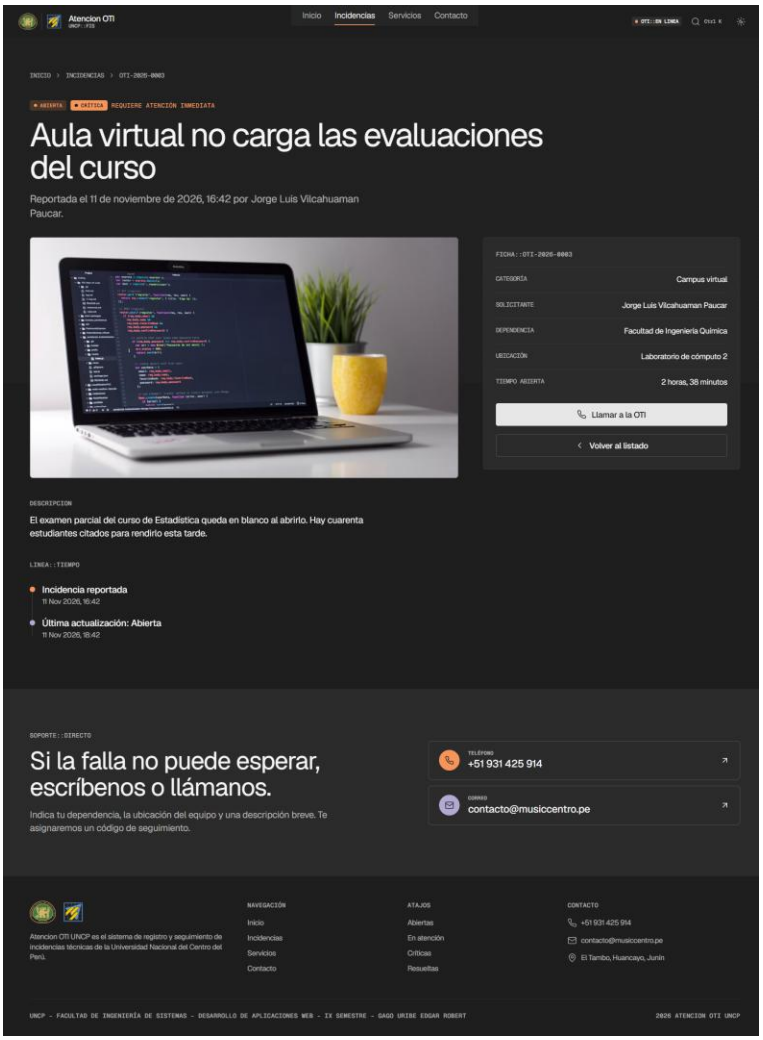

Inicio con Tema Claro




Lista de Incidencias




Visualización de Detalle Incidencia



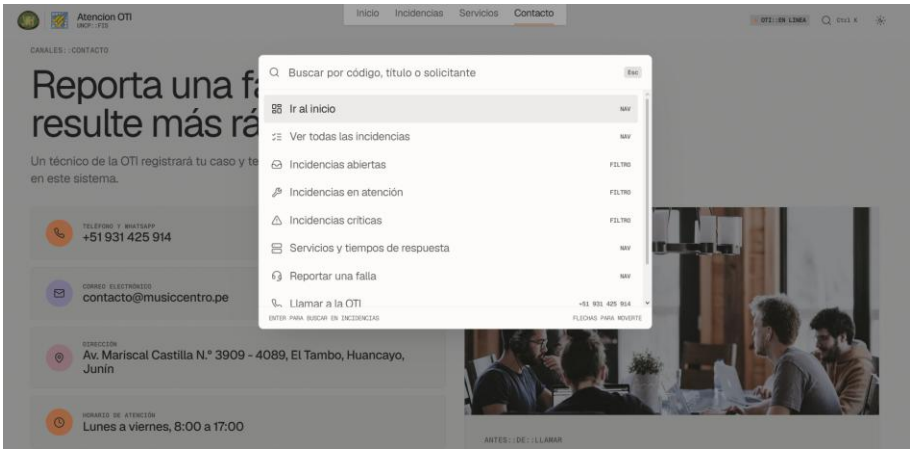

Catálogo de Servicios



Página de Contacto




Paleta de Comandos




**Enlace de GitHub del Proyecto:**
https://github.com/E5gar/GuiaPracticaSemana11